

Paralelní implementace Conwayovy Hry života

Denis Zuev

ZUE0008

2026

Seminární práce — PDS

Contents

1	Úvod do problému	2
1.1	Volba jazyka: Python a problém GIL	2
2	State of the Art	2
3	Popis implementovaných algoritmů	3
3.1	Sekvenční implementace (Sparse-Set)	3
3.2	Vektorizovaná implementace (NumPy)	3
3.3	Paralelní implementace (Multiprocessing + Shared Memory)	4
3.4	GPU implementace (OpenCL)	4
4	Experimenty a měření	5
4.1	Podmínky měření	5
4.2	Hypotézy	5
4.3	Výsledky	6
4.3.1	Malý pattern: diehard2500 (100×100)	6
4.3.2	Střední pattern: rpentominoequivalents (392×214)	6
4.3.3	Velký pattern: pp8primecalculator (2044×1236)	7
4.3.4	Porovnání napříč patterny	8
4.4	Vyhodnocení hypotéz	8
4.5	Analýza škálování	9
5	Závěr	9

1 Úvod do problému

Conwayova Hra života (Game of Life) [1] je dvourozměrný celulární automat definovaný na mřížce $G \in \{0,1\}^{H \times W}$. Každá buňka má 8 sousedů (Mooreovo okolí) a v každém kroku se aplikují pravidla:

- **Přežití:** živá buňka s 2 nebo 3 živými sousedy zůstává živá.
- **Zrození:** mrtvá buňka s přesně 3 živými sousedy se stává živou.
- **Smrt:** ve všech ostatních případech buňka umírá.

Z výpočetního hlediska se jedná o **2D stencilový výpočet** — operaci, kde nový stav každé buňky závisí pouze na jejím okolí. Tato vlastnost činí problém *embarrassingly parallel* — nezávislé buňky mohou být vyhodnocovány souběžně bez synchronizace [2].

Cílem práce je implementovat a experimentálně porovnat čtyři přístupy k výpočtu: sekvenční (čistý Python), vektorizovaný (NumPy), multiprocessing se sdílenou pamětí a GPU (OpenCL), a kvantifikovat dosažené zrychlení.

1.1 Volba jazyka: Python a problém GIL

Implementace je v jazyce **Python** (CPython 3.11+). Python je interpretovaný jazyk s dynamickým typováním, jehož hlavní nevýhodou pro paralelní výpočty je **Global Interpreter Lock (GIL)** [8] — mutex, který zajišťuje, že v daném okamžiku vykonává Python bytecode pouze jedno vlákno. To znamená, že:

- `threading.Thread` a `ThreadPoolExecutor` **neposkytují skutečný CPU paralelismus** pro Python kód — vlákna jsou přepínána kooperativně, nikdy neběží současně.
- GIL je dočasně uvolněn během volání C-extension funkcí (např. NumPy operace), což umožňuje *omezenou* paralelizaci při vektorizovaném výpočtu.
- Pro skutečný CPU paralelismus v Pythonu je nutné použít **procesy** (`multiprocessing`), kde každý proces má vlastní GIL a vlastní kopii interpretu.

Z tohoto důvodu byla paralelní implementace v této práci realizována pomocí `multiprocessing` se sdílenou pamětí (`shared_memory`), nikoli pomocí vláken. Sdílená paměť eliminuje nutnost serializace (pickle) velkých NumPy polí mezi procesy.

2 State of the Art

Paralelizace celulárních automatů je dobře prostudovaná oblast. Obvyklé přístupy zahrnují:

Dekompozice domény. Mřížka se rozdělí na bloky (řádkové, sloupcové nebo 2D), každý blok je přiřazen jednomu výpočetnímu vláknu/procesu. Při hranicích bloků je nutná výměna *ghost cells* (překryvových řádků) [3].

SIMD vektorizace. Knihovny jako NumPy provádějí aritmetické operace na celých polích pomocí SIMD instrukcí (AVX2/SSE), čímž eliminují režii interpretu Pythonu [4].

GPU výpočty. Každá buňka je mapována na jeden work-item. Frameworky: CUDA [5], OpenCL [6], Vulkan Compute. Pro Game of Life je výpočet pamětově omezený (memory-bound) — ALU je nevytížen, limitujícím faktorem je propustnost paměti.

HashLife. Algoritmicky odlišný přístup, založený na memoizaci quadtree podpatternů. Dosahuje exponenciálního zrychlení pro regulární patterny, ale má vysokou pamětovou náročnost [7].

Tato práce implementuje první tři přístupy (vektorizace, multiprocessing, GPU) a porovnává je se sekvenční implementací.

3 Popis implementovaných algoritmů

3.1 Sekvenční implementace (Sparse-Set)

Sekvenční implementace pracuje s množinou živých buněk místo iterace přes celou mřížku:

1. Extrahovat souřadnice živých buněk: $L = \{(r, c) \mid G[r, c] = 1\}$
2. Sestrojit množinu kandidátů: $C = \bigcup_{(r, c) \in L} N(r, c) \cup \{(r, c)\}$, kde $N(r, c)$ je Mooreovo okolí
3. Pro každého kandidáta $(r, c) \in C$ spočítat $n = |\{(r', c') \in N(r, c) \cap L\}|$
4. Aplikovat pravidla: nová živá buňka pokud $n = 3$, nebo $n = 2$ a $(r, c) \in L$

Složitost: $O(|L| \cdot 9)$ na krok. Efektivní pro řídké patterny ($|L| \ll H \cdot W$).

3.2 Vektorizovaná implementace (NumPy)

Využívá operace nad celými poli — žádná Python smyčka:

1. Převést mřížku na `int8` pole P
2. Vytvořit padded verzi: $P' = \text{pad}(P, 1, \text{mode}=\text{constant})$
3. Spočítat sousedy jako součet 8 posunutých řezů:

$$N[i, j] = \sum_{\substack{(dr, dc) \in \{-1, 0, 1\}^2 \\ (dr, dc) \neq (0, 0)}} P'[i+dr+1, j+dc+1]$$

4. Aplikovat pravidla vektorově:

$$G_{\text{next}} = (N = 3) \vee (G_{\text{curr}} \wedge N = 2)$$

Složitost: $O(H \times W)$ na krok, ale s konstantním faktorem řádově $100\times$ menším díky SIMD.

3.3 Paralelní implementace (Multiprocessing + Shared Memory)

Rozšiřuje vektorizovaný přístup o datovou dekompozici po řádcích s použitím procesů místo vláken, čímž zcela obchází Python GIL:

1. Alokovat sdílenou paměť (`multiprocessing.shared_memory`) pro padded vstup a výstupní mřížku
2. Spustit T perzistentních workerů (kde $T = \text{os.cpu_count}()$)
3. V každém kroku:
 - (a) Hlavní proces zapíše aktuální mřížku do sdílené paměti
 - (b) `Barrier.wait()` uvolní workery
 - (c) Každý worker provede vektorizovaný výpočet na svém řádkovém bloku a zapíše výsledek přímo do sdílené výstupní paměti
 - (d) Druhý `Barrier.wait()` signalizuje dokončení
4. Hlavní proces zkopíruje výsledek ze sdílené paměti do gridu

Klíčové rozdíly oproti `ThreadPoolExecutor`:

- Procesy běží nezávisle — žádný GIL
- Sdílená paměť eliminuje serializaci (pickle) velkých polí
- Perzistentní workery — žádný per-step overhead na fork/spawn
- `Barrier` synchronizace má minimální latenci

Složitost: $O\left(\frac{H \times W}{T}\right)$ na worker + režie na barrier synchronizaci a kopii do sdílené paměti.

3.4 GPU implementace (OpenCL)

Každá buňka mřížky je mapována na jeden OpenCL work-item:

1. Alokovat dva buffery (`src`, `dst`) velikosti $H \times W$ bytů na GPU
2. Nahrát počáteční stav do `src`
3. V každém kroku spustit kernel s global size ($\lceil W/16 \rceil \cdot 16$, $\lceil H/16 \rceil \cdot 16$) a work-group size (16, 16)
4. Kernel pro pozici (x, y) : spočítat sousedy čtením z `src`, zapsat výsledek do `dst`
5. Prohodit `src` a `dst` (pointer swap, žádná kopie dat)
6. Host sync (`enqueue_copy` zpět do RAM) pouze na konci simulace

Úplný zdrojový kód kernelu:

Listing 1: OpenCL kernel — gameOfLife.cl

```

1  __kernel void gameOfLifeStep(
2      __global const uchar* src,
3      __global uchar* dst,
4      const int width,
5      const int height)
6  {
7      int x = get_global_id(0);
8      int y = get_global_id(1);
9      if (x >= width || y >= height) return;
10
11     int neighbors = 0;
12     for (int dy = -1; dy <= 1; dy++) {
13         for (int dx = -1; dx <= 1; dx++) {
14             if (dx == 0 && dy == 0) continue;
15             int nx = x + dx, ny = y + dy;
16             if (nx >= 0 && nx < width && ny >= 0 && ny < height)
17                 if (src[ny * width + nx] > 0) neighbors++;
18         }
19     }
20
21     int idx = y * width + x;
22     uchar alive = src[idx];
23     dst[idx] = (alive > 0) ? (neighbors == 2 || neighbors == 3)
24                     : (neighbors == 3);
25 }

```

4 Experimenty a měření

4.1 Podmínky měření

Parametr	Hodnota
CPU	AMD Phoenix3 APU, 16 logických jader
GPU	Integrovaná GPU (Mesa rusticl, OpenCL 3.0)
Patterny	diehard2500 (100×100), rpentominoequivalents (392×214), pp8primecalculator (2044×1236, ≈2.5M buněk)
Kroky	1000
Měření	Opakované 3×, prezentovány průměrné hodnoty
OS	Linux (kernel 6.x)
Python	3.11+

Table 1: Podmínky experimentu

4.2 Hypotézy

Před provedením experimentů formulujeme následující hypotézy:

- H1:** Vektorizace (NumPy) přinese zrychlení alespoň $10\times$ oproti čistému Pythonu díky eliminaci režie interpretru.
- H2:** Multiprocessing se sdílenou pamětí přinese zrychlení blízke $T/2$ oproti vektorizované verzi pro dostatečně velké mřížky, ale pro malé mřížky bude pomalejší kvůli režii.
- H3:** GPU implementace dosáhne nejvyššího zrychlení ($> 100\times$) díky masivnímu SIMD paralelismu.

4.3 Výsledky

4.3.1 Malý pattern: diehard2500 (100×100)

Metoda	Čas (ms)	Avg (ms/krok)	Zrychlení
Pure Python (Sparse)	1 876.7	1.88	$1.0\times$
NumPy (Vectorized)	30.6	0.03	$61.3\times$
Parallel (Multiprocessing, 16p)	396.3	0.40	$4.7\times$
GPU (OpenCL)	31.3	0.03	$60.0\times$

Table 2: diehard2500 — 1000 kroků, 100×100 mřížka

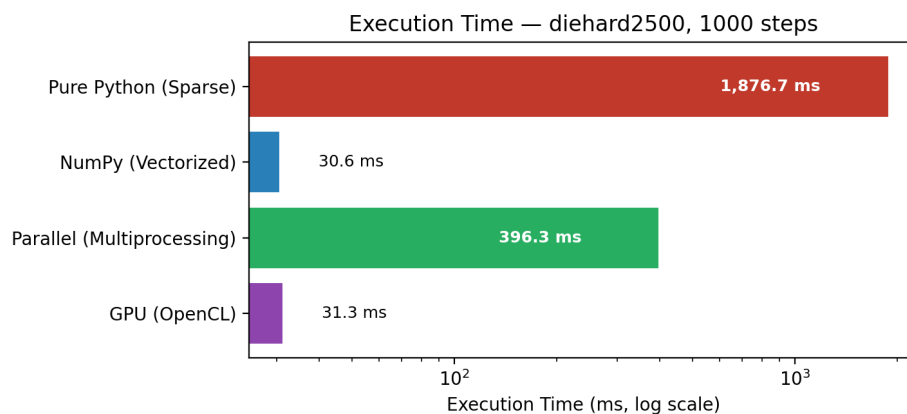


Figure 1: Doba výpočtu — diehard2500 (logaritmické měřítko)

4.3.2 Střední pattern: rpentominoequivalents (392×214)

Metoda	Čas (ms)	Avg (ms/krok)	Zrychlení
Pure Python (Sparse)	5 493.9	5.49	$1.0\times$
NumPy (Vectorized)	91.7	0.09	$59.9\times$
Parallel (Multiprocessing, 16p)	400.7	0.40	$13.7\times$
GPU (OpenCL)	48.8	0.05	$112.7\times$

Table 3: rpentominoequivalents — 1000 kroků, 392×214 mřížka

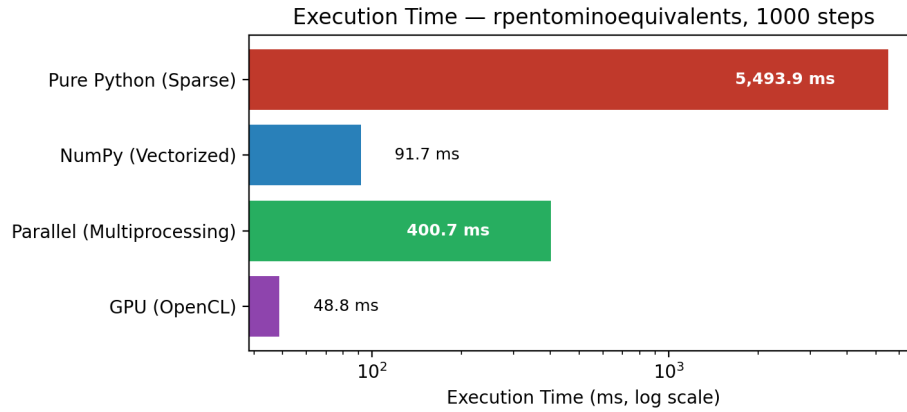


Figure 2: Doba výpočtu — rpentominoequivalents (logaritmické měřítko)

4.3.3 Velký pattern: pp8primecalculator (2044×1236)

Metoda	Čas (ms)	Avg (ms/krok)	Zrychlení
Pure Python (Sparse)	74 843.3	74.84	1.0×
NumPy (Vectorized)	4 199.5	4.20	17.8×
Parallel (Multiprocessing, 16p)	1 840.5	1.84	40.7×
GPU (OpenCL)	333.9	0.33	224.1×

Table 4: pp8primecalculator — 1000 kroků, ≈ 2.5 M buněk

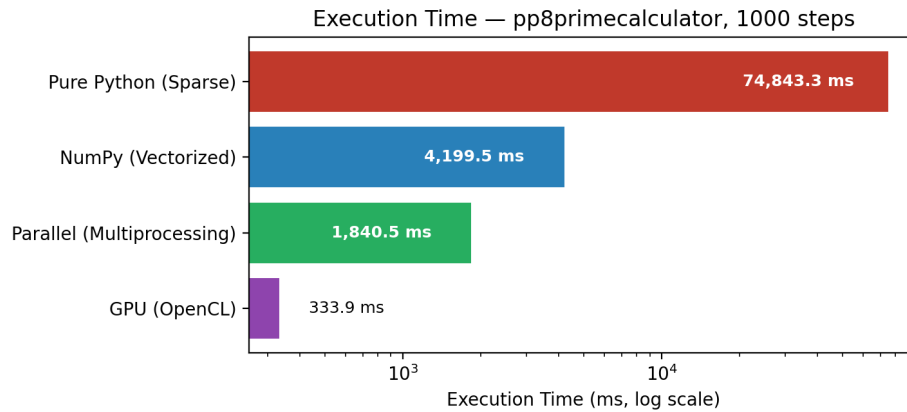


Figure 3: Doba výpočtu — pp8primecalculator (logaritmické měřítko)

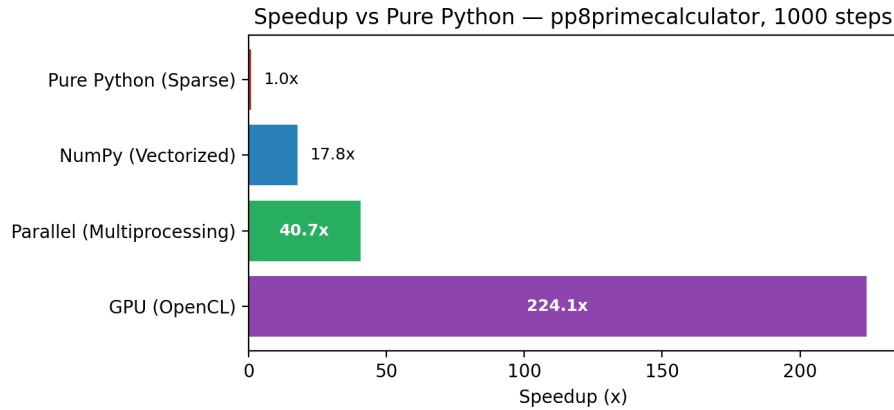


Figure 4: Zrychlení oproti sekvenční implementaci — pp8primecalculator

4.3.4 Porovnání napříč patterny

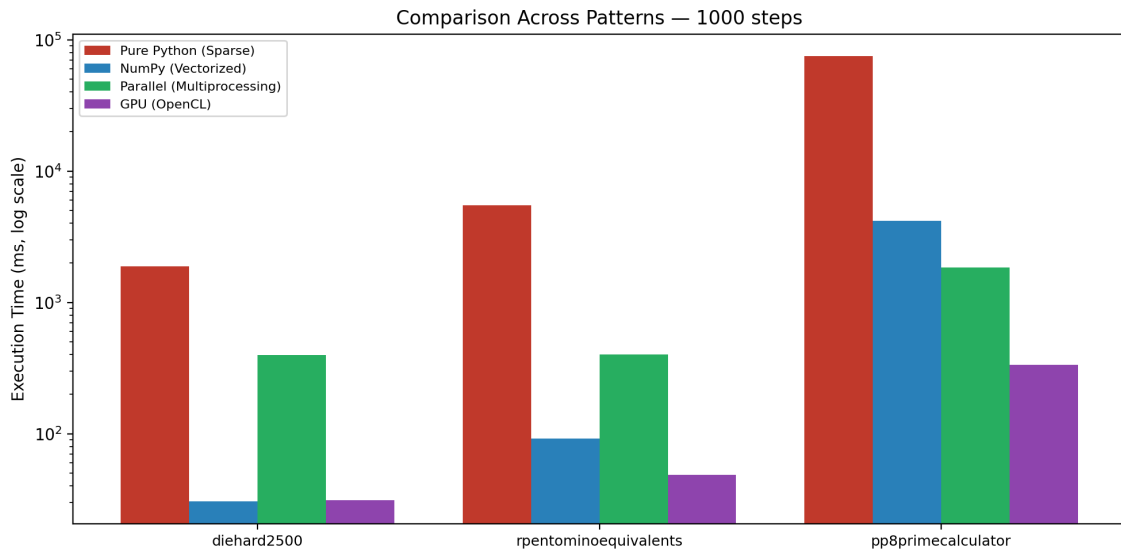


Figure 5: Porovnání dob výpočtu napříč patterny (logaritmické měřítko)

4.4 Vyhodnocení hypotéz

H1 — potvrzena. Vektorizace dosáhla 18–61× zrychlení napříč patterny v závislosti na velikosti mřížky, což převyšuje předpokládaných 10×. Na menších mřížkách (diehard2500) je poměr vyšší (61×) díky nižší absolutní režii.

H2 — potvrzena. Multiprocessing se sdílenou pamětí vykazuje závislost na velikosti mřížky, jak bylo očekáváno:

- **Malá mřížka** (100×100): multiprocessing je 13× *pomalejší* než vectorized — režie barrier synchronizace dominuje
- **Střední mřížka** (392×214): stále 4.4× *pomalejší* než vectorized

- **Velká mřížka** (2044×1236): $2.3 \times$ *rychlejší* než vectorized — výpočetní práce per-worker dostatečně převáží režii

Paralelní efektivita pro velkou mřížku (16 procesů):

$$\eta = \frac{S_p}{T} = \frac{2.3}{16} \approx 14\%$$

Nízká efektivita je způsobena per-step kopírováním mřížky do sdílené paměti a barrier latencí.

H3 — potvrzena. GPU dosáhlo $224 \times$ zrychlení oproti baseline na velkém patternu. GPU je konzistentně nejrychlejší napříč všemi velikostmi mřížek.

4.5 Analýza škálování

Přechod (pp8)	Faktor	Vysvětlení
Sequential → Vectorized	$17.8 \times$	Eliminace Python smyček; SIMD operace
Vectorized → Parallel	$2.3 \times$	Datová dekompozice; obejití GIL
Parallel → GPU	$5.5 \times$	SIMT paralelismus; žádný per-step sync
Sequential → GPU	$224 \times$	Celkové zrychlení

Table 5: Analýza škálování — pp8primecalculator

5 Závěr

Práce demonstrovala progresivní paralelizaci 2D stencilového výpočtu (Conwayova Game of Life) pomocí čtyř přístupů s rostoucí úrovní paralelismu.

Hlavní zjištění:

1. Samotná vektorizace (NumPy) přináší $18 \times$ zrychlení na velké mřížce eliminací režie interpretru — jedná se o nejjednodušší cestu k výraznému zrychlení na CPU.
2. Multiprocessing se sdílenou pamětí přidává další $2.3 \times$ na velkých mřížkách, ale je kontraproduktivní pro malé vstupy kde režie synchronizace dominuje.
3. GPU (OpenCL) dosahuje $224 \times$ zrychlení oproti baseline a je jasným vítězem pro velké mřížky.

Future work:

- Porovnání s HashLife algoritmem pro regulární patterny
- Testování na diskrétní GPU (vyšší memory bandwidth)
- Škálování: měření pro různé velikosti mřížek (100×100 až 10000×10000)
- Implementace pomocí CUDA pro srovnání s OpenCL
- Adaptivní volba backendu podle velikosti mřížky

References

- [1] Gardner, M. *Mathematical Games — The fantastic combinations of John Conway’s new solitaire game “life”*. Scientific American, 223(4):120–123, 1970.
- [2] Mattson, T. et al. *Patterns for Parallel Programming*. Addison-Wesley, 2004.
- [3] Quinn, M. J. *Parallel Programming in C with MPI and OpenMP*. McGraw-Hill, 2003.
- [4] Harris, C. R. et al. Array programming with NumPy. *Nature*, 585:357–362, 2020.
- [5] Nickolls, J. et al. Scalable parallel programming with CUDA. *ACM Queue*, 6(2):40–53, 2008.
- [6] Stone, J. E. et al. OpenCL: A parallel programming standard for heterogeneous computing systems. *Computing in Science & Engineering*, 12(3):66–73, 2010.
- [7] Gosper, R. W. Exploiting regularities in large cellular spaces. *Physica D*, 10(1–2):75–80, 1984.
- [8] Beazley, D. Understanding the Python GIL. *PyCon 2010*, Atlanta, GA, 2010. <https://dabeaz.com/python/UnderstandingGIL.pdf>